Lab 6: Tiến trình và tín hiệu trong Linux
`fork`, `exec`, `wait`, zombie/orphan, `sigaction`

Hướng dẫn thực hành Lập trình hệ thống

Trường Đại học Công nghệ - ĐHQGHN

Ngày 29 tháng 10 năm 2026

Mục lục

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm `exec`
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

Nội dung

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm exec
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

Mục tiêu buổi thực hành

Sau buổi hôm nay, sinh viên phải làm được (theo mục 1 của Tutorial)

1. **Nhìn thấy tiến trình:** phân biệt chương trình/tiến trình; đọc PID, PPID, cột `STAT`; định vị tiến trình trong cây bằng `ps`, `pstree`, `/proc`.
2. **Tạo và thay thế tiến trình đúng cách:** `fork` (3 nhánh), không gian địa chỉ riêng nhưng *offset file chung*, bộ đệm stdio bị sao chép; `exec` không trả về khi thành công; `_exit` trong con.
3. **Đồng bộ và thu dọn:** `waitpid` + macro `W...`; `WNOHANG`; tạo và giải thích zombie, orphan; không để zombie sống quá lâu.
4. **Xử lý tín hiệu an toàn:** `kill`, `sigaction`; handler chỉ đặt cờ `volatile sig_atomic_t`; kết thúc có kiểm soát với `SIGINT`/`SIGTERM`; thu dọn bằng `SIGCHLD`.

Sản phẩm cần nộp

`bai1.c` ... `bai12.c` (Bài 3: `bai3a.c`, `bai3b.c`) + tuỳ chọn `mr1_stdio_fork.c`, `mr2_wnohang.c`, `mr3_sigchld.c` và `Makefile`; `make` **sạch cảnh báo** với cờ của môn; `valgrind --leak-check=full --track-fds=yes` sạch (cả tiến trình con); báo cáo: output thật của `ps`/`pstree` (Bài 1, 9, 10) và phần *giải thích* của Bài 4, 5, 8, MR1.

Trả lời câu hỏi của tuần trước: bộ đệm của FILE * ở đâu, lớn bao nhiêu?

```c
/* bufsize.c: 10000 byte, khong co '\n' */
for (int i = 0; i < 10000; i++)
    putchar('a' + i % 26);
```

```
$ strace -e trace=write ./bufsize > /dev/null
write(1, "...", 4096) = 4096
write(1, "...", 4096) = 4096
write(1, "...", 1808) = 1808
$ strace -e trace=write ./bufsize      # ra tty
write(1, "...", 1024) = 1024     (x9)
write(1, "...", 784) = 784
```

- Bộ đệm là một mảng **trong bộ nhớ của tiến trình** (cấp phát lúc ghi lần đầu), *không* nằm trong kernel.
- Cỡ = `st_blksize` của fd 1: **4096** với file/pipe, **1024** với terminal (pty) – đo thật trên Ubuntu 24.04.
- Đẩy xuống kernel khi: *đầy*; gặp `'\n'` (chỉ khi ra terminal); `fflush`; `exit()`/`return` từ `main`.

Hệ quả hôm nay

Bộ đệm thuộc *không gian địa chỉ* ⇒ `fork` sao chép nó (MR1); `_exit` và tín hiệu giết tiến trình làm *mất* nó (slide sau).

...và `kill -9` ngay sau `printf("Xong")` thì sao?

```c
/* xong.c */
printf(argc > 1 ? "Xong\n" : "Xong");
kill(getpid(), SIGKILL);  /* nhu kill -9 */
return 0;
```

```
$ ./xong; echo "[rc=$?]"        # tty, khong '\n'
bash: line 1:   391 Killed     ./xong
[rc=137]
$ ./xong n; echo "[rc=$?]"      # tty, co '\n'
Xong
bash: line 1:   392 Killed     ./xong n
[rc=137]
$ ./xong n > f.txt; wc -c f.txt # file, co '\n'
bash: line 1:   393 Killed     ./xong n > f.txt
0 f.txt
```

- `SIGKILL` kết thúc tiến trình **ngay trong kernel**: không chạy `exit()`, không đẩy bộ đệm stdio.
- Ra terminal + `'\n'`: đã đẩy theo dòng → còn; ra file: bộ đệm đầy-khối → **mất** dù có `'\n'`.
- Mã thoát của shell: $137 = 128 + 9$.

Bài học dùng cả buổi

Dữ liệu quan trọng: `fflush` (hoặc ghi thẳng bằng `write`) *trước* khi có thể bị `fork`, `exec`, `_exit` hay tín hiệu cắt ngang. Với `SIGTERM`/`SIGINT` ta có thể *bắt* và kết thúc có kiểm soát (Bài 11, 12); với `SIGKILL` thì không.

Lab 6 trong môn học – và tài liệu của buổi hôm nay

Học ở Lab 6	**Gặp lại ở các tuần sau**
`fork` + `exec` + `waitpid`	Pipeline `ls | wc -l` (Lab 7, Bài 9)
fd kế thừa qua `fork`, chung offset	`pipe()` trước `fork`, đóng đầu không dùng (Lab 7)
fd giữ nguyên qua `exec`	`dup2` để chuyển hướng stdout/stdin (Lab 7, Bài 6–8)
Bộ đệm stdio bị sao chép	Dữ liệu "lặp" khi ghi vào pipe (Lab 7)
`SIGCHLD`, `SIGPIPE`, `sigaction`	Ghi vào pipe không còn reader (Lab 7, Bài 13)
`EINTR`, `SA_RESTART`	`read` trên pipe/FIFO bị tín hiệu cắt ngang
Tiến trình độc lập bộ nhớ	Luồng chia sẻ bộ nhớ (Lab 9)

Tài liệu chính

`Lab6_Tutorial.pdf` (16 tr.) + `Lab6_Exercises.pdf` (12 bài + 3 mở rộng). Slide đi theo đúng thứ tự Tutorial: tiến trình → định danh, cây, trạng thái → `fork` → kết thúc, `exec` → `wait` → zombie/orphan → tín hiệu, `sigaction`, `SIGCHLD`.

Đã nghe ở lớp lý thuyết (SP07)

SP07 *Linux Process Handling* trùng gần hết khái niệm ⇒ chỉ *nhắc nhanh*; thời gian dồn cho *quan sát bằng máy* và các bẫy chỉ lộ ra khi chạy thật.

Bản đồ: mục nào của Tutorial dùng cho bài nào

Mục của Tutorial	Bài tập dùng	Slide nhấn mạnh thêm (chạy thật)
§2–4 Tiến trình, PID/PPID, `ps`, `pstree`, trạng thái, `watch`	Bài 1	`/proc/<pid>`; cây trên WSL khác Ubuntu
§5 `fork`: 3 giá trị trả về, không gian địa chỉ, COW, fd, stdio	Bài 2, 3, 4, 5; MR1	Thống kê thứ tự cha/con; `strace -f` chung offset; `BeforeBefore`
§6–7 `exit`/`_exit`, họ `exec`	Bài 7, 8	PID không đổi qua `exec`; `execvp` dò `PATH`; 126 hay 127
§8 `wait`/`waitpid`, macro, `WNOHANG`, `EINTR`	Bài 3, 6; MR2	Giá trị thô của `status` (`status_demo`)
§9 Zombie, orphan	Bài 9, 10	`Z+ <defunct>`; subreaper `/init` của WSL
§10–12 Tín hiệu, `kill`, `sigaction`, `SIGCHLD`	Bài 11, 12; MR3	`-std=c11` giấu `sigaction`; `pause` và khe hở; `sigsuspend`
§13–14 Mẫu tổng hợp, lỗi thường gặp	mọi bài	`Makefile`, `valgrind --track-fds` cho cả con

Cách học trong buổi

Mỗi mục: *ví dụ của Tutorial* chạy thật → quan sát bằng công cụ ở terminal thứ hai → bẫy → điểm dừng là một bài tập.

Nội dung

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm exec
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

Chương trình và tiến trình – nhắc nhanh (Tutorial §2, SP07)

Chương trình	File thực thi trên đĩa, thụ động, *không* có PID
Tiến trình	Một lần thực thi, kernel quản lý, có PID riêng

Thành phần (§2.2): mã lệnh, dữ liệu toàn cục/tĩnh, heap, stack, **bảng fd**, thư mục làm việc, người dùng, **thiết lập tín hiệu**.

Ngữ cảnh (§2.3): trạng thái CPU (PC, SP, thanh ghi, cờ) + thông tin kernel giữ (PID, PPID, bản đồ bộ nhớ, bảng fd, cwd, tín hiệu) → *context switch*.

Mỗi dòng ở đây là một "thuộc tính của tiến trình"

- Bảng fd: Lab 5 (`/proc/self/fd`) – hôm nay: *được sao chép* khi `fork` (Bài 5).
- Thư mục làm việc: Lab 5, Bài 8 (`linuxcd`) – vì sao `cd` phải là lệnh dựng sẵn.
- Heap, stack, data: Lab 3 – hôm nay: *mỗi tiến trình một bản* (Bài 4).
- Thiết lập tín hiệu: hôm nay (Bài 11, 12) – được kế thừa qua `fork`, *bị đặt lại* một phần qua `exec`.

Quan sát được tất cả

`ls /proc/<PID>/`: `status`, `fd/`, `cwd`, `maps`, `stat...`

PID, PPID, ps và pstree – Bài 1 chạy thật

```c
/* bai1.c (rut gon) */
printf("PID = %ld\n", (long)getpid());
printf("PPID = %ld\n", (long)getppid());
fflush(stdout);
unsigned left = 10;
while (left > 0)
    left = sleep(left);
```

```
$ ./bai1                        # terminal 1
PID = 376
PPID = 372
$ ps -o pid,ppid,stat,cmd -p 376 -p 372
    PID    PPID STAT CMD
    372     369 Ss+  bash /tmp/t1.sh
    376     372 S+   ./bai1
```

- PPID của `bai1` là **shell** đã chạy nó: shell cũng `fork` + `exec` (sẽ thấy ở Bài 7).
- `pid_t` in bằng `%ld` + ép `(long)`: chuẩn không nói `pid_t` rộng bao nhiêu.
- `S+`: `S` ngủ (đang `sleep`), `+` thuộc nhóm *foreground*; `Ss+`: `s` = trưởng phiên (shell).

```
$ pstree -A -p 372
bash(372)-+-bai1(376)
          `-pstree(380)
```

Trên WSL

Gốc cây là `/init` của WSL (`Relay(...)---bash`), không phải `systemd(1)`. Ở phòng lab Ubuntu: `systemd(1)---...---bash---bai1`. PID mỗi lần chạy một khác.

Cột STAT và `watch` (Tutorial §4)

Ký hiệu	Tên	Gặp trong lab ở đâu
`R`	Running/Runnable	Vòng lặp bận (không `sleep`)
`S`	Sleeping	`sleep`, chờ `read` từ bàn phím, `waitpid`
`D`	Uninterruptible	I/O đĩa – hiếm khi bắt được
`T`	Stopped	`Ctrl+Z`; `kill -STOP`; gdb dừng
`Z`	Zombie	Bài 9 (`<defunct>`)

Ký tự phụ: `+` foreground, `s` trưởng phiên (shell), `l` đa luồng, `<`/`N` ưu tiên.

```
# quan sat lien tuc, 0.5 s mot lan
watch -n 0.5 'ps -o pid,ppid,stat,cmd -p 376'
# thu chuyen trang thai bang tin hieu
kill -STOP 376      # S+ -> T+  (do that)
kill -CONT 376      # T+ -> S+
```

Trạng thái đổi rất nhanh

Hai lần `ps` liên tiếp có thể khác nhau; `R` chỉ thấy khi chương trình thật sự dùng CPU. Đừng kết luận từ *một* lần chạy `ps`.

Nội dung

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm exec
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

fork(): một lời gọi, hai lần trả về (Tutorial §5.1–5.2)

```c
/* bai2.c */
pid_t pid = fork();

if (pid == -1) {          /* 1. loi */
    perror("fork");
    return 1;
}
if (pid == 0) {           /* 2. trong con */
    printf("Child: PID=%ld PPID=%ld\n",
           (long)getpid(), (long)getppid());
    return 0;
}
/* 3. trong cha: pid = PID cua con */
printf("Parent: PID=%ld child=%ld\n",
       (long)getpid(), (long)pid);
return 0;
```

```
$ ./bai2
Parent: PID=632 child=633
Child: PID=633 PPID=632
```

- Cả hai tiến trình chạy tiếp từ *ngay sau* `fork`; chỉ giá trị trả về khác nhau.
- `-1`: *không* có con (`EAGAIN`: hết giới hạn `ulimit -u`; `ENOMEM`).
- Con lấy PID của cha bằng `getppid()`; cha *chỉ* biết PID của con qua giá trị trả về ⇒ phải giữ lại.

Đừng quên nhánh -1

`if (pid) {cha} else {con}` coi `-1` là "cha" rồi `waitpid(-1, ...)` chờ *bất kỳ* con nào!

Thứ tự cha–con: đo 200 lần (Bài 2, Bài 3a)

```
for i in $(seq 200); do ./bai3a | head -1; done \
    | sort | uniq -c
```

`bai3a` chạy với	cha trước	con trước
stdout là terminal	199	1
stdout là pipe (`| head`)	0	200
1 CPU (`taskset -c 0`), pipe	0	200

Đo trên máy soạn (WSL2, 4 nhân, kernel 6.18).

- Ra terminal: cha *gần như luôn* in trước (Linux không cho con chạy trước) – nhưng **không phải luôn luôn**: $1/200$.
- Ra pipe: *con luôn trước*! Không phải do lập lịch: dòng của cha nằm trong **bộ đệm** cho đến khi cha thoát – mà `bai3a` thu dọn con *sau* khi in, nên cha thoát sau con.

Kết luận của đề, được đo lại

"Không được giả định cha hoặc con chạy trước" – và thứ tự *dòng chữ* còn phụ thuộc *bộ đệm*, không chỉ thứ tự *chạy*.

Không gian địa chỉ riêng, cùng địa chỉ ảo (Tutorial §5.3–5.4; Bài 4)

```
$ ./bai4
Truoc  PID=693 idata=1000 (&idata=0x570cd9fb9010) istack=150 (&istack=0x7fff52111334)
Con    PID=694 idata=3000 (&idata=0x570cd9fb9010) istack=450 (&istack=0x7fff52111334)
Cha    PID=693 idata=1000 (&idata=0x570cd9fb9010) istack=150 (&istack=0x7fff52111334)
```

- Con đổi `idata`, `istack` ($\times 3$); cha vẫn thấy 1000 và 150.
- **Cùng** địa chỉ `0x570c...9010` ở cả hai: đó là địa chỉ *ảo*; mỗi tiến trình có bảng trang riêng ánh xạ tới trang vật lý riêng.
- Lab 3: `&istack` trên stack (`0x7fff...`), `&idata` ở vùng data – giữ nguyên bố cục.

Copy-on-Write

Sau `fork`, hai bên *dùng chung* trang vật lý, đánh dấu chỉ đọc. Lần ghi đầu (`idata *= 3`) gây lỗi trang $\rightarrow$ kernel sao một bản riêng cho bên ghi. Vì thế `fork` rẻ dù tiến trình lớn – và `fork` rồi `exec` ngay gần như không sao chép gì.

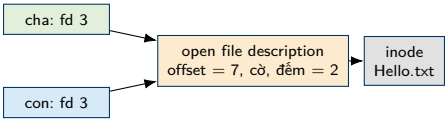
Hệ quả

Biến chung giữa cha và con *không tồn tại* ở đây. Muốn trao đổi dữ liệu: pipe, FIFO, bộ nhớ chung (Lab 7).

Bảng fd được sao chép – offset thì *dùng chung* (Tutorial §5.5; Bài 5)

```
$ strace -f -e trace=openat,lseek,read,close,clone,wait4 ./bai5
602 openat(AT_FDCWD, "Hello.txt", O_RDONLY) = 3
602 clone(...) = 603
602 wait4(603,  <unfinished ...>
603 lseek(3, 7, SEEK_SET)             = 7
603 close(3)                          = 0
603 +++ exited with 0 +++
602 <... wait4 resumed>[{WIFEXITED(s) && ...
602 read(3, "What is your name?", 4096) = 18
602 read(3, "", 4096)                 = 0
602 close(3)                          = 0
```

Con `lseek` rồi `close(3)` *của nó*; cha `read(3)` vẫn bắt đầu từ byte 7.



- Lab 5: `open` hai lần → hai description, offset riêng. Ở đây: *một* `open` trước `fork` → một description.
- `close` trong con chỉ giảm *số tham chiếu*; description sống đến khi fd cuối cùng đóng.

Bộ đệm stdio cũng bị "sao chép" (Tutorial §5.6; Mở rộng 1)

```c
/* mr1_stdio_fork.c */
printf("Before");     /* khong co '\n' */
if (do_flush)         /* ./mr1_stdio_fork -f */
    fflush(stdout);
pid_t pid = fork();
...
return 0;             /* ca hai exit() */
```

	terminal	`> out.txt`
không `fflush`	`BeforeBefore`	`BeforeBefore`
có `fflush` (`-f`)	`Before`	`Before`

- Không `'\n'` ⇒ "Before" còn trong bộ đệm *ở cả hai chế độ* ⇒ in hai lần **kể cả ra terminal**. Đề hỏi "terminal khác file thế nào?" – với chuỗi này: *không khác*.
- Khác biệt terminal/file chỉ hiện khi chuỗi *có* `'\n'` (ví dụ `afterfork.c` của SP07): terminal đẩy theo dòng → 1 lần; file → 2 lần.

Quy tắc

`fflush(stdout)` (hoặc `fflush(NULL)`) ngay *trước* `fork`. Mọi lời giải của tuần này đều làm vậy.

Nội dung

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm exec
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

exit() và _exit() (Tutorial §6)

`exit(s)`	Gọi hàm atexit, **đẩy và đóng** mọi `FILE *`, rồi `_exit(s)`
`return s`	Từ `main`: giống `exit(s)`
`_exit(s)`	System call: kết thúc ngay, kernel đóng fd, **bộ đệm stdio mất**
Tín hiệu	`SIGKILL`, `SIGSEGV`...: như `_exit`, cha thấy `WIFSIGNALED`

Chỉ 8 bit thấp của s đến được cha: `_exit(256)` → cha đọc 0 (slide `status_demo`).

Khi nào dùng _exit trong con?

Con *kế thừa* bộ đệm và hàm atexit của cha. Nếu con gọi exit, những thứ đó chạy *thêm một lần* → dữ liệu in lặp, file tạm bị xoá hai lần... Nên: nhánh con sau exec thất bại dùng `_exit(127)`.

Bẫy ngược: _exit nuốt output của chính con

Nếu con `printf` rồi `_exit` mà stdout là file/pipe, dòng đó **biến mất**. Bài 4 phải `fflush(stdout)` trước `_exit(0)` – Tutorial chưa nói.

Họ exec: thay chương trình, giữ tiến trình (Tutorial §7.1–7.3)

Hàm	Đối số	Tìm file
execl	danh sách	đường dẫn
execv	argv[]	đường dẫn
execlp	danh sách	theo PATH
execvp	argv[]	theo PATH
execle	danh sách	đường dẫn + envp
execve	argv[]	đường dẫn + envp

l/v: list/vector; p: PATH; e: môi trường. Năm hàm đầu đều gọi execve (system call duy nhất).

```c
char *argv[] = {"ls", "-l", NULL};  /* NULL cuoi! */
execvp(argv[0], argv);
perror("execvp");   /* chi toi day khi THAT BAI */
_exit(127);

execlp("ls", "ls", "-l", (char *)NULL);
/*        ^file ^argv[0]               */
```

- Thành công: **không trả về** – mã cũ không còn. Mọi dòng sau exec là xử lý lỗi.
- argv[0] là *tên chương trình* theo quy ước, không phải đối số đầu.
- execlp(..., NULL): ép (char *)NULL – NULL có thể là 0 kiểu int trong hàm có ....

PID không đổi qua exec; execvp dò PATH thế nào

```c
/* exec_pid.c - nhanh con */
printf("Con truoc exec: PID=%s PPID=%ld\n",
       me, (long)getppid());
fflush(stdout);
execlp("ps", "ps", "-o", "pid,ppid,cmd",
       "-p", me, (char *)NULL);
```

```
$ ./exec_pid
Con truoc exec: PID=401 PPID=400
    PID    PPID CMD
    401     400 ps -o pid,ppid,cmd -p 401
Cha: PID=400
```

```
$ strace -f -e trace=clone,execve,wait4 ./bai7 >/dev/null
442 clone(...) = 443
442 wait4(443,  <unfinished ...>
443 execve("/usr/local/sbin/ls", ...) = -1 ENOENT
443 execve("/usr/local/bin/ls", ...) = -1 ENOENT
443 execve("/usr/sbin/ls", ...) = -1 ENOENT
443 execve("/usr/bin/ls", ["ls", "-l"], ...) = 0
443 +++ exited with 0 +++
442 <... wait4 resumed>... = 443
442 --- SIGCHLD {si_code=CLD_EXITED, ...} ---
```

Tiến trình 401 chạy `exec_pid` rồi chạy ps – *cùng* PID, cùng PPID. `execvp` thử từng thư mục của `PATH`; `fork` trong glibc là `clone`; cha nhận `SIGCHLD` khi con kết thúc.

Cái gì còn, cái gì mất sau exec (Tutorial §7.5, SP07)

Giữ nguyên

PID, PPID, thư mục làm việc, người dùng; **fd đang mở** (trừ fd có `O_CLOEXEC`/`FD_CLOEXEC`); tín hiệu đang **bỏ qua** (`SIG_IGN`) vẫn bị bỏ qua; mặt nạ tín hiệu; tín hiệu đang chờ.

Thay mới

Mã lệnh, data, heap, stack, **bộ đệm stdio** (thuộc chương trình cũ – chưa `fflush` là mất); tín hiệu đang **bắt bằng handler** → trở về `SIG_DFL` (địa chỉ handler không còn nghĩa).

Vì sao điều này quan trọng

- fd còn qua exec là nền của chuyển hướng: mở file, dup2 vào fd 1, rồi exec – chương trình mới ghi vào file mà không biết (Lab 7, Bài 6).
- `SIG_IGN` còn qua exec: shell bỏ qua `SIGINT` phải *đặt lại* `SIG_DFL` trong con trước exec (SP07, "Ctrl+C behavior in a shell").

Tutorial §7.5 ghi "đưa về mặc định đối với *nhiều* tín hiệu"

Chính xác hơn: *mọi* tín hiệu đang bắt bằng handler; tín hiệu đang bỏ qua thì *giữ nguyên*.

Nội dung

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm exec
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

wait và waitpid (Tutorial §8.1, §8.3)

```c
#include <sys/wait.h>
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

`pid > 0`	đúng con có PID đó
`pid == -1`	con bất kỳ (`wait(&s)` = `waitpid(-1, &s, 0)`)
`options = 0`	chặn đến khi con đổi trạng thái
`WNOHANG`	không chặn: trả 0 nếu chưa có gì
trả về > 0	PID của con đã thu dọn
trả về -1	lỗi: `ECHILD` (không còn con), `EINTR`

```c
/* bai3b.c: cha chi in SAU khi con xong */
static int wait_child(pid_t pid, int *status) {
    pid_t r;
    do {
        r = waitpid(pid, status, 0);
    } while (r == -1 && errno == EINTR);
    return (r == pid) ? 0 : -1;
}
```

```
$ ./bai3b       (3/3 lan, terminal hay pipe)
I am the child
I am the parent
```

`waitpid` vừa **đồng bộ** (cha dừng tới khi con xong) vừa **thu dọn** (xoá bản ghi của con trong kernel).

Trạng thái kết thúc: dùng macro, đừng tự tách bit (Tutorial §8.2; Bài 6)

```
$ ./status_demo
_exit(7)        status=0x0700  WIFEXITED   WEXITSTATUS=7
_exit(256)      status=0x0000  WIFEXITED   WEXITSTATUS=0
raise(SIGTERM)  status=0x000f  WIFSIGNALED WTERMSIG=15 (Terminated)
*NULL = 1       status=0x008b  WIFSIGNALED WTERMSIG=11 (Segmentation
      fault)
kill -KILL      status=0x0009  WIFSIGNALED WTERMSIG=9 (Killed)
```

```c
/* bai6.c */
if (WIFEXITED(status))
    printf("Child exit code = %d\n", WEXITSTATUS(status));
else if (WIFSIGNALED(status))
    printf("Child killed by signal %d\n", WTERMSIG(status));
```

- `status` là số *đã mã hoá*: mã thoát nằm ở byte thứ hai (0x0700), số tín hiệu ở 7 bit thấp, bit 0x80 = có core dump (0x008b).
- Cách mã hoá là chi tiết của hệ thống ⇒ **chỉ** dùng macro.
- Chỉ đọc `WEXITSTATUS` khi `WIFEXITED` đúng; `WTERMSIG` khi `WIFSIGNALED` đúng.
- Shell đổi thành `$?`: thoát bình thường → mã; bị tín hiệu → 128+số (137, 139).

WNOHANG và EINTR (Tutorial §8.4–8.5; Mở rộng 2)

```c
/* mr2_wnohang.c - vong lap cua cha */
for (;;) {
    pid_t r = waitpid(pid, &status, WNOHANG);
    if (r == -1) {
        if (errno == EINTR) continue;
        perror("waitpid"); return 1;
    }
    if (r == pid) break;      /* da thu don */
    printf("Child is still running\n");
    fflush(stdout);
    sleep(1);                 /* lam viec khac */
}
```

```
$ ./mr2_wnohang
Child is still running
Child is still running
Child is still running
Child is still running
Child is still running
Child 436 exited, code = 3
```

- 3 kết quả: >0 xong, 0 chưa, −1 lỗi – xử lý *cả ba*.
- EINTR: một tín hiệu (có handler, không SA_RESTART) cắt ngang lời gọi đang chặn → gọi lại.
- 5 hay 6 dòng "still running" tuỳ lúc con thức dậy – đừng kiểm tra bằng số dòng.

Nội dung

1 Mở đầu: từ Lab 5 sang Lab 6

2 Tiến trình, định danh và trạng thái

3 Tạo tiến trình với `fork()`

4 Kết thúc tiến trình và họ hàm exec

5 Chờ tiến trình con: họ `wait`

6 Zombie và orphan

7 Tín hiệu và `sigaction()`

8 Mẫu tổng hợp, lỗi thường gặp và tra cứu

9 Bài tập thực hành

10 Tổng kết

Zombie: đã chết, chưa được thu dọn (Tutorial §9.1, §9.3; Bài 9)

```
$ ./bai9                                # terminal 1
Parent PID = 358, child PID = 360
Terminal 2: ps -o pid,ppid,stat,cmd -p 360
$ ps -o pid,ppid,stat,cmd -p 360        # terminal 2
    PID    PPID STAT CMD
    360     358 Z+   [bai9] <defunct>
...  20 giay sau, terminal 1:
Reaped child 360
$ ps -o pid,ppid,stat,cmd -p 360        # terminal 2
    PID    PPID STAT CMD
$ echo $?
1
```

- Kernel giữ *bản ghi kết thúc* (PID, `status`, thời gian CPU) cho đến khi cha `wait`.
- Zombie không chạy, không chiếm bộ nhớ chương trình – nhưng **giữ một PID**. Server quên `wait` → hết PID → `fork` trả `EAGAIN`.
- `kill -9` zombie *vô tác dụng*: nó đã chết. Chỉ cha `wait` hoặc cha chết (zombie được "nhận nuôi" rồi thu dọn).

Bài 9 có thời hạn

Nhấn `Ctrl+C` trong lúc cha ngủ: cả nhóm foreground nhận `SIGINT`, cha chết, zombie được tiến trình nhận nuôi thu dọn – đề yêu cầu *không* để zombie quá lâu.

Orphan: cha chết trước, con được nhận nuôi (Tutorial §9.2, §9.4; Bài 10)

```
$ ./bai10 > b10.out; sleep 4; cat b10.out
Parent PID=377 exits now
Child PID=378, PPID ban dau=377
Child PID=378, PPID sau 3 giay=273
$ ps -o pid,ppid,stat,cmd -p 273
    PID    PPID STAT CMD
    273     272 S    /init
$ ps -o pid,ppid,cmd -p 1
    PID    PPID CMD
      1       0 /sbin/init
```

Trên WSL, PPID mới là `/init` của *phiên* WSL (273), không phải PID 1.

- Khi cha chết, kernel chuyển con cho **subreaper** gần nhất (`prctl(PR_SET_CHILD_SUBREAPER)`), nếu không có thì PID 1.
- Ubuntu desktop: thường `systemd --user`; container/WSL: tiến trình init của nó. Vì vậy đề viết "không yêu cầu PPID mới phải luôn bằng 1".
- Orphan *vẫn chạy*, khác zombie (*đã chết*). Người nhận nuôi luôn `wait` ⇒ orphan không thành zombie lâu dài.

Chạy trên terminal

Dấu nhắc $ hiện lại ngay khi *cha* thoát; hai dòng của con in *sau* dấu nhắc. Không phải lỗi.

Nội dung

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm exec
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

Tín hiệu, lệnh `kill`, hàm `kill()`/`raise()` (Tutorial §10.1–10.4)

Tín hiệu	Nguồn	Mặc định
SIGINT	Ctrl+C	kết thúc – bắt được
SIGTERM	kill PID	kết thúc – bắt được
SIGKILL	kill -KILL	kết thúc – **không** bắt/bỏ qua
SIGSTOP	kill -STOP	dừng – **không** bắt/bỏ qua
SIGCHLD	con đổi trạng thái	**bỏ qua**
SIGSEGV	truy cập sai bộ nhớ	kết thúc + core
SIGPIPE	ghi vào pipe hết reader	kết thúc (Lab 7)

Mỗi tín hiệu: *mặc định*, *bỏ qua* (SIG_IGN) hoặc *handler*.

```
kill -TERM 2412    # = kill 2412
kill -s INT 2412
kill -l            # liet ke ten va so
```

```
#include <signal.h>
int kill(pid_t pid, int sig);  /* 0 / -1 */
int raise(int sig);            /* cho chinh minh */
```

- "kill" là *gửi tín hiệu*, không nhất thiết giết.
- Dùng tên SIGTERM, không dùng số: SIGCHLD là 17 trên x86/ARM, khác trên MIPS, SPARC.
- Tín hiệu chỉ mang *một số*: cùng loại đến dồn dập có thể bị **gộp** (MR3).

sigaction(): cài handler đúng cách (Tutorial §10.5, §11.1–11.2)

```c
/* bai11.c / bai12.c */
static volatile sig_atomic_t stop = 0;

static void on_int(int sig) {
    (void)sig;
    stop = 1;           /* CHI dat co */
}

static int install(int sig, void (*h)(int)) {
    struct sigaction sa = {0};
    sa.sa_handler = h;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    return sigaction(sig, &sa, NULL);
}
```

`sa_handler`	hàm, `SIG_DFL` hoặc `SIG_IGN`
`sa_mask`	tín hiệu bị *chặn thêm* khi handler chạy
`sa_flags`	`SA_RESTART`, ...
`oldact`	nhận thiết lập cũ (hoặc `NULL`)

`signal()` cũ: ngữ nghĩa khác nhau giữa các hệ (có hệ đặt lại `SIG_DFL` sau mỗi lần nhận, có hệ không khởi động lại system call) ⇒ môn dùng `sigaction`.

Lỗi biên dịch đầu tiên của tuần

Với `-std=c11` mà thiếu `-D_POSIX_C_SOURCE=200809L`: `struct sigaction` "chưa biết". Slide sau.

Bẫy: ví dụ §11.4 của Tutorial với cờ của môn

```
$ gcc -Wall -Wextra -std=c11 -pedantic tut114.c -o tut114      # vi du 11.4, chep nguyen van
tut114.c:13:12: error: variable 'sa' has initializer but incomplete type
tut114.c:13:28: warning: excess elements in struct initializer
tut114.c:13:22: error: storage size of 'sa' isn't known
tut114.c:15:5: warning: implicit declaration of function 'sigemptyset' [-Wimplicit-function-declaration]
tut114.c:17:9: warning: implicit declaration of function 'sigaction' [-Wimplicit-function-declaration]
$ gcc -Wall -Wextra -std=c11 -pedantic -D_POSIX_C_SOURCE=200809L tut114.c -o tut114
$                                                              # sach, khong canh bao
```

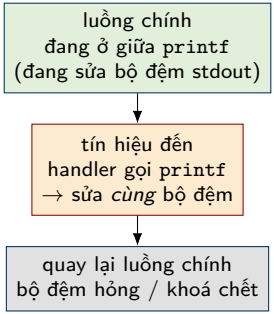
- `-std=c11` = C *thuần*: glibc giấu mọi thứ của POSIX (`sigaction`, `kill`, `sigset_t`...).
- Không phải thiếu `#include`! Thêm `<signal.h>` lần nữa không giúp gì.
- Tutorial biên dịch bằng `gcc -Wall -Wextra` (mặc định gnu17) nên không gặp.

Cách sửa (đã nằm sẵn trong `Makefile` từ Lab 5)

`CPPFLAGS = -D_POSIX_C_SOURCE=200809L`
hoặc dòng đầu file: `#define _POSIX_C_SOURCE 200809L` (trước mọi `#include`).

Thử cả bộ lời giải khi thiếu macro: `bai11`, `bai12`, `mr3` lỗi `sigaction`; `bai2`, `bai10`, `mr1` lỗi `unknown type name 'pid_t'` (chỉ có `<stdio.h>`, `<unistd.h>`); các bài có `<sys/wait.h>` lại qua được – vì header đó tự khai báo `pid_t`.

Handler chỉ đặt cờ – vì sao? (Tutorial §11.3)



- Handler chạy *xen vào bất kỳ lệnh máy nào* của luồng chính. `printf`, `malloc`, `exit` dùng cấu trúc dữ liệu chung → không *async-signal-safe*.
- An toàn: gán `volatile sig_atomic_t`; `write`, `_exit`, `waitpid`... (danh sách trong `man 7 signal-safety`).
- `volatile`: buộc vòng lặp `while (!stop)` đọc lại biến mỗi lần; `sig_atomic_t`: ghi trong một bước.

Mẫu của cả tuần

Handler đặt cờ → luồng chính thấy cờ → in thông báo, đóng file, xoá file tạm, `return 0`.

Chờ cờ thế nào? sleep, pause và khe hở

```
while (!stop)          /* Tutorial, Bai 11-12 */
    sleep(1);          /* tre toi da 1 s: an toan */

while (!flag)          /* NGUY HIEM */
    pause();           /* tin hieu den giua 2 dong? */
```

```
$ ./pause_race     # mo phong: raise() ngay truoc pause()
Alarm clock        # treo, alarm(3) phai giet
$ echo $?
142
```

- `sleep` bị tín hiệu cắt ngang thì trả về *sớm* → vòng lặp thấy cờ ngay; tệ nhất trễ 1 giây.
- `pause`: nếu tín hiệu đến *sau* khi kiểm tra cờ, *trước* khi `pause` → ngủ chờ tín hiệu *kế tiếp* – có thể mãi mãi.
- Cách đúng khi cần ngủ đến khi có tín hiệu: **chặn** tín hiệu, kiểm tra cờ, rồi `sigsuspend(&old)` (mở chặn + ngủ trong *một* bước) – dùng ở MR3.

SA_RESTART và SIGCHLD (Tutorial §11.5, §12; Mở rộng 3)

```
$ ./mr3_sigchld               # con ngu 1 s va 3 s
Started child 367 (sleep 1 s)
Started child 368 (sleep 3 s)
Reaped 367, exit code = 10
Lan thuc day 1: thu don 1 con
Reaped 368, exit code = 11
Lan thuc day 2: thu don 1 con
All 2 children reaped
$ ./mr3b                      # hai con cung ngu 1 s
...
Lan thuc day 1: thu don 2 con     (4/4 lan chay)
```

- Hai con chết gần nhau → *một* lần thức dậy, *hai* con: tín hiệu cùng loại đang chờ bị gộp. ⇒ luôn `while (waitpid(-1, &s, WNOHANG) > 0)`.
- `SA_RESTART`: `read`, `waitpid`... bị `SIGCHLD` cắt ngang được kernel tự gọi lại – *không* áp dụng cho `sleep`, `pause`, `sigsuspend`.

Thứ tự trong ví dụ §12 của Tutorial

Tutorial vét `waitpid` *rồi mới* `child_changed = 0`: con chết giữa hai bước bị xoá cờ → có thể chờ mãi. Đặt cờ về 0 **trước** khi vét.

Bẫy khi thử nghiệm: tiến trình nền bỏ qua SIGINT

```
$ cat test.sh
./bai12 > b12.out &        # bai12 chi bat SIGTERM
kill -INT $!               # mong: bai12 chet
wait $!                    # thuc te: TREO MAI
$ grep -E 'SigIgn|SigCgt' /proc/$(pgrep -x bai12)/status
SigIgn: 0000000000000006
SigCgt: 0000000000004000
```

SigIgn bit 1, 2 → tín hiệu 2 (SIGINT), 3 (SIGQUIT) bị *bỏ qua*; SigCgt bit 14 → tín hiệu 15 (SIGTERM) có handler.

- Shell *không tương tác* (script) chạy lệnh với & thì đặt SIGINT, SIGQUIT = SIG_IGN cho nó – và SIG_IGN được giữ qua exec.
- bai11 vẫn dừng được vì sigaction của nó *ghi đè* SIG_IGN.
- /proc/<pid>/status: xem thiết lập tín hiệu của bất kỳ tiến trình nào.

Khi viết script kiểm thử

Thử SIGINT bằng Ctrl+C thật, hoặc qua pseudo-terminal; dùng SIGTERM cho tiến trình nền.

Nội dung

1 Mở đầu: từ Lab 5 sang Lab 6

2 Tiến trình, định danh và trạng thái

3 Tạo tiến trình với `fork()`

4 Kết thúc tiến trình và họ hàm exec

5 Chờ tiến trình con: họ `wait`

6 Zombie và orphan

7 Tín hiệu và `sigaction()`

8 Mẫu tổng hợp, lỗi thường gặp và tra cứu

9 Bài tập thực hành

10 Tổng kết

Mẫu fork + exec + waitpid (Tutorial §13) và Makefile

```
fflush(stdout);                 /* 0. bo dem rong */
pid_t pid = fork();             /* 1 */
if (pid == -1) { perror("fork"); return 1; }
if (pid == 0) {                 /* 2. con */
    execvp(argv[0], argv);
    perror("execvp");
    _exit(127);
}
int status;                     /* 3. cha */
pid_t r;
do r = waitpid(pid, &status, 0);
while (r == -1 && errno == EINTR);
if (r == -1) { perror("waitpid"); return 1; }
if (WIFEXITED(status)) ...      /* 4. giai ma */
```

```
# Makefile - Lab 6
CPPFLAGS = -D_POSIX_C_SOURCE=200809L
CFLAGS   = -Wall -Wextra -std=c11 -pedantic \
           -g -O0 -MMD -MP
PROGS = bai1 bai2 bai3a bai3b ... bai12 \
        mr1_stdio_fork mr2_wnohang mr3_sigchld
all: $(PROGS)
$(PROGS): %: %.o
clean:
    rm -f $(PROGS) *.o *.d
```

Đây là "khung xương" của mọi shell: Lab 7 chỉ chèn thêm pipe/dup2 giữa bước 1 và 2.

Valgrind với chương trình có `fork`

```
$ valgrind --leak-check=full --track-fds=yes ./bai5
==598== FILE DESCRIPTORS: 3 open (3 std) at exit.   <- con
==598== All heap blocks were freed -- no leaks ...
==598== ERROR SUMMARY: 0 errors from 0 contexts
What is your name?
==596== FILE DESCRIPTORS: 3 open (3 std) at exit.   <- cha
==596== All heap blocks were freed -- no leaks ...
==596== ERROR SUMMARY: 0 errors from 0 contexts
```

Bản đầu của `bai5` (con `_exit` không `close(fd)`): tiến trình con báo `4 open (3 std)`.

- Valgrind theo cả cha và con sau `fork` (không cần `--trace-children`; cờ đó dành cho `exec`).
- Hai báo cáo, phân biệt bằng PID ==598==.
- fd "rò" trong con sắp `_exit` không hại (kernel đóng), nhưng giữ thói quen Lab 5: đóng trên mọi đường ra.

Kết quả cả bộ: 10 chương trình có `fork` đều `0 errors`, không rò bộ nhớ, không rò fd.

Lỗi thường gặp (Tutorial §14) → triệu chứng → công cụ

Triệu chứng	**Nguyên nhân thường gặp**	**Công cụ / cách sửa**
Dòng in *hai lần* khi chuyển hướng ra file	Bộ đệm stdio chưa đẩy trước `fork`	`fflush(stdout)` trước `fork`; MR1
Dòng in của con *biến mất* khi ra file	Con `printf` rồi `_exit`	`fflush` trước `_exit`
"Chương trình chạy hai lần" sau `exec` lỗi	Con quên `_exit` sau `exec` thất bại	`perror`; `_exit(127)` ngay sau `exec`
Cha đọc nhầm mã thoát (1792 thay vì 7)	In thẳng `status`	`WIFEXITED`/`WEXITSTATUS`
`<defunct>` trong `ps`; `fork: EAGAIN`	Không `wait`	`waitpid`; `SIGCHLD` + vòng `WNOHANG`
Con "mất" một nửa dữ liệu file	fd chung offset với cha	Mở file *sau* `fork` nếu cần offset riêng
`storage size of 'sa' isn't known`	`-std=c11` giấu POSIX	`-D_POSIX_C_SOURCE=200809L`
Treo khi chờ tín hiệu	pause sau kiểm tra cờ (khe hở)	`sigprocmask` + `sigsuspend`
`kill -INT` không có tác dụng	Tiến trình nền của script: `SIGINT` bị bỏ qua	`/proc/<pid>/status`; dùng `SIGTERM`
Chương trình treo, không rõ chờ gì	Đang chặn trong `wait4`/`read`...	`strace -p <pid>`, `ps -o stat,wchan`

Tra cứu: man và công cụ của tuần

```
man 2 fork      man 2 execve    man 3 exec
man 2 waitpid   man 2 _exit     man 3 exit
man 2 kill      man 2 sigaction man 7 signal
man 7 signal-safety             man 2 sigsuspend
man 1 ps        man 1 pstree    man 5 proc
```

WSL bản tối giản không có man: cài sudo apt install man-db manpages-dev hoặc tra https://man7.org/linux/man-pages/.

```
ps -o pid,ppid,pgid,stat,cmd   # them nhom
pstree -A -p $$                # cay tu shell
cat /proc/<pid>/status         # PPid, SigIgn...
ls -l /proc/<pid>/fd           # fd ke thua
strace -f -e trace=process ./prog
kill -l; kill -STOP/-CONT <pid>
```

strace -f: theo cả tiến trình con; -e trace=process: chỉ clone, execve, wait4, exit...

Ba chỗ cần đọc kỹ trong Tutorial (đã chạy thật)

```
/* Tutorial §3.1, chep nguyen van */
printf("PID = %ld\\n", (long)getpid());
printf("PPID = %ld\\n", (long)getppid());
```

```
$ ./tut31
PID = 589\nPPID = 297\n$
```

`\\n` in ra *hai ký tự* `\` và `n`, không xuống dòng. Sửa: `\n`.

1. §3.1: `\\n` (bên trái).
2. §11.4 (và mọi ví dụ `sigaction`): không biên dịch với cờ của môn nếu thiếu `_POSIX_C_SOURCE`.
3. §12: đặt `child_changed = 0` *sau* vòng `waitpid` → có thể mất một con.

Thêm: §5.3 ví dụ không kiểm tra `fork` trả -1 (trái với §14, lỗi 1); §6.2 chưa nói con phải `fflush` trước `_exit`; §7.5 "nhiều tín hiệu" (slide trước). Danh mục đầy đủ: hai slide phụ lục ở cuối.

Nội dung

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm exec
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

Kế hoạch 90 phút và các điểm dừng

Thời lượng	Nội dung hướng dẫn (theo Tutorial)	Sinh viên làm (điểm dừng)
0–7′	Mục tiêu; câu hỏi dẫn: bộ đệm `FILE *` (`bufsize`, `xong`)	Chạy `xong` 3 cách
7–17′	**Tiến trình**: PID/PPID, `ps`, `pstree`, `STAT` (§2–4)	**Bài 1** (2 terminal)
17–37′	`fork`: 3 nhánh, thứ tự, không gian địa chỉ, fd chung, bộ đệm (§5)	**Bài 2**, **Bài 4**, **Bài 5**
37–50′	`_exit`, `exec` (§6–7); PID giữ nguyên, `PATH`, 127	**Bài 7**, **Bài 8**
50–60′	`waitpid`, macro, `WNOHANG` (§8)	**Bài 3**, **Bài 6**
60–68′	**Zombie, orphan** (§9)	**Bài 9** (khởi động, quan sát `Z`)
68–84′	**Tín hiệu**: `kill`, `sigaction`, cờ `sig_atomic_t`, `-std=c11` (§10–11)	**Bài 12**
84–90′	`SIGCHLD` (§12), `Makefile`, Valgrind; yêu cầu nộp bài	`make sạch`
Sau 90′	Tự làm: Bài 10, 11; Mở rộng 1–3 (MR3 dùng `sigsuspend`); báo cáo	

Quy tắc của buổi thực hành

Gõ tay, không copy–paste từ PDF. `make` **sạch cảnh báo** với cờ của môn; `fflush(stdout)` trước mọi `fork`; kiểm tra *mọi* `fork`/`exec`/`waitpid`/`sigaction`; luôn mở **hai terminal** – một chạy, một quan sát.

Bài 1 – Quan sát PID và PPID

```c
#define ALIVE_SECONDS 10

int main(void) {
    printf("PID = %ld\n", (long)getpid());
    printf("PPID = %ld\n", (long)getppid());
    fflush(stdout);   /* in ngay ca khi ra pipe */

    unsigned left = ALIVE_SECONDS;
    while (left > 0)  /* sleep tra ve so giay */
        left = sleep(left);   /* con lai neu bi */
    return 0;                 /* tin hieu cat */
}
```

Kiểm chứng (output thật ở mục 2): PID/PPID khớp `ps -o pid,ppid,stat,cmd -p <PID>`; `STAT = S+`; `pstree -p` cho thấy `bash(PPID)---bai1(PID)`; `./bai1 | cat`: hai dòng vẫn hiện *ngay* (nhờ `fflush`), không phải sau 10 giây.

- `sleep(10)` một lần cũng đạt đề; vòng `left = sleep(left)` đúng cả khi bị tín hiệu cắt ngang (`kill -STOP/-CONT`).
- Báo cáo: dán output *thật* của `ps` và `pstree` – PID của bạn khác ví dụ.

Bài 2 – `fork()` với đủ ba nhánh

```c
pid_t pid = fork();
if (pid == -1) {
    perror("fork");
    return 1;
}
if (pid == 0) {
    printf("Child: PID=%ld PPID=%ld\n",
           (long)getpid(), (long)getppid());
    return 0;
}
printf("Parent: PID=%ld child=%ld\n",
       (long)getpid(), (long)pid);
return 0;
```

```
$ ./bai2
Parent: PID=632 child=633
Child: PID=633 PPID=632
```

Kiểm chứng: chạy 5 lần qua pseudo-terminal – 5/5 lần dòng cha trước; `child=` của cha bằng `PID=` của con; `PPID` của con bằng `PID` của cha.

Đôi khi chỉ thấy một dòng

Cha `return` trước khi con kịp in, dấu nhắc hiện ra; dòng của con in *sau* dấu nhắc (hoặc mất nếu terminal đã đóng – gặp thật khi chạy qua `script`). Cha *không* `wait` là theo đúng đề; PPID con in ra có thể đã là tiến trình nhận nuôi nếu cha thoát rất sớm.

Bài 3 – Có và không có `waitpid()`

```c
/* bai3a.c */
if (pid == 0) {
    printf("I am the child\n");
    return 0;
}
printf("I am the parent\n"); /* khong wait TRUOC */
waitpid(pid, NULL, 0);       /* thu don SAU */
return 0;

/* bai3b.c */
if (wait_child(pid, &status) == -1) {
    perror("waitpid");
    return 1;
}
printf("I am the parent\n"); /* sau khi con xong */
```

200 lần	terminal	pipe
bai3a: cha trước	199	0
bai3a: con trước	1	200
bai3b: con trước	3/3	3/3

Giả định: đề chỉ cấm `wait` *trước* khi cha in $\Rightarrow$ bai3a vẫn thu dọn con *sau* khi in (không để zombie/orphan, dấu nhắc không chen vào giữa).

Kết luận đúng và kết luận sai

Sai: "bai3a luôn cha trước" (chỉ đúng 199/200, và sai hẳn khi ra pipe). Đúng: chỉ bai3b *đảm bảo* thứ tự – bằng đồng bộ, không bằng may mắn.

Bài 4 – Dữ liệu sau `fork()`

```c
static int idata = 1000;            /* data */

int main(void) {
    int istack = 150;               /* stack */
    show("Truoc", istack);
    fflush(stdout);
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) {
        idata *= 3;
        istack *= 3;
        show("Con", istack);
        fflush(stdout);             /* truoc _exit! */
        _exit(0);
    }
    /* waitpid co vong EINTR */
    show("Cha", istack);            /* 1000, 150 */
}
```

Kiểm chứng: output ở mục 3 (`Con` 3000/450, `Cha` 1000/150, cùng địa chỉ ảo); `./bai4 > o4.txt`: vẫn đủ 3 dòng, đúng thứ tự. **Nhận xét cần viết**: con có bản sao (logic) của toàn bộ không gian địa chỉ; COW tách trang khi con ghi; địa chỉ in ra là *ảo*.

Hai cách làm hỏng Bài 4

(1) Bỏ `fflush` trước `fork` → ra file: dòng "Truoc" xuất hiện **hai lần**. (2) Bỏ `fflush` trước `_exit` → ra file: dòng "Con" **biến mất**. Ra terminal thì cả hai lỗi đều *không lộ*.

Bài 5 – Vị trí tệp dùng chung sau `fork()`

```c
int fd = open(path, O_RDONLY);   /* TRUOC fork */
...
pid_t pid = fork();
if (pid == 0) {
    int rc = 0;
    if (lseek(fd, CHILD_OFFSET, SEEK_SET) == (off_t)-1) {
        perror("lseek");
        rc = 1;
    }
    close(fd);           /* chi fd cua con */
    _exit(rc);
}
/* cha: waitpid; kiem tra WEXITSTATUS == 0 */
long long n = copy_rest(fd);  /* read tu offset 7 */
...
close(fd);
```

```
$ ./bai5
What is your name?
$ ./bai5 khongco.txt
open khongco.txt: No such file or directory
$ printf 'abc' > short.txt; ./bai5 short.txt
(khong con gi de doc tu offset 7)
```

Kiểm chứng: ví dụ đề; file không có (mã 1); file ngắn hơn 7 byte (`read` trả 0 ngay); `strace -f` ở mục 3; Valgrind sạch cả hai tiến trình.

- `copy_rest` = vòng `read`/`write` của Lab 5 (đọc thiếu, `EINTR`).
- Cha kiểm tra con *thật sự* đặt được offset (mã thoát 0) trước khi đọc.

Bài 5 – phân tích: "chung offset" nghĩa là gì, và khi nào *không*?

- Chung: **offset**, cờ trạng thái (`O_APPEND`, `O_NONBLOCK`) – nằm trong open file description.
- Riêng: *số* fd, cờ `FD_CLOEXEC` – nằm trong bảng fd của từng tiến trình.
- Nếu con `open` lại `Hello.txt` *sau* `fork`: description mới, offset riêng – cha đọc từ 0.

Vì sao thiết kế như vậy?

Shell chạy `(echo a; echo b) > f`: hai tiến trình con ghi *nối tiếp* nhau vào `f` nhờ chung offset – nếu mỗi con có offset riêng, `b` sẽ ghi đè `a`. Pipe ở Lab 7 dựa trên đúng cơ chế kế thừa này.

Bẫy

Cha và con cùng `read` một fd chung *đồng thời*: mỗi bên nhận một phần ngẫu nhiên của file.

Bài 6 – Giải mã trạng thái kết thúc

```c
#define CHILD_CODE 7

static void print_status(int status) {
    if (WIFEXITED(status))
        printf("Child exit code = %d\n",
               WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("Child killed by signal %d\n",
               WTERMSIG(status));
    else
        printf("Child status = 0x%x (khac)\n",
               (unsigned)status);
}
/* main: fork; con _exit(CHILD_CODE);
   cha waitpid (vong EINTR) roi print_status */
```

```
$ ./bai6
Child exit code = 7
```

Kiểm chứng: ví dụ đề; các nhánh còn lại thử bằng `status_demo` (mục 5): `_exit(256)` $\rightarrow$ 0, `SIGSEGV` $\rightarrow$ 11, `SIGKILL` $\rightarrow$ 9.

- Đề chỉ yêu cầu `WIFEXITED`; danh mục cuối đề yêu cầu cả `WIFSIGNALED`/`WTERMSIG` $\Rightarrow$ lời giải xử lý đủ.
- `print_status` tách riêng: dùng lại ở Bài 7, 8, MR2, MR3 và Lab 7.

Bài 7 – Chạy `ls -l` bằng execvp

```c
static pid_t spawn(char *const argv[]) {
    pid_t pid = fork();
    if (pid == 0) {
        execvp(argv[0], argv);
        perror("execvp");       /* chi khi THAT BAI */
        _exit(127);
    }
    return pid;                 /* -1 neu fork loi */
}

int main(void) {
    char *argv[] = {"ls", "-l", NULL};
    fflush(stdout);
    pid_t pid = spawn(argv);
    ... waitpid ...
    if (WIFEXITED(status))
        return WEXITSTATUS(status);  /* ma cua ls */
    ...
}
```

```
$ ./bai7
total 8
-rw-r--r-- 1 noname noname   25 Oct 29 09:43 Hello.txt
-rwxr-xr-x 1 noname noname 1090 Oct 29 09:43 bai7.c
$ ls -l
total 8
-rw-r--r-- 1 noname noname   25 Oct 29 09:43 Hello.txt
-rwxr-xr-x 1 noname noname 1090 Oct 29 09:43 bai7.c
```

Kiểm chứng: `diff <(./bai7) <(ls -l)` rỗng; `strace -f` ở mục 4; mã thoát của `bai7` = mã của `ls`.

- Đây chính là việc shell làm khi bạn gõ `ls -l`.
- Cha *không* cần biết `ls` in gì – output đi thẳng ra fd 1 được kế thừa.

Bài 8 – exec thất bại

```c
char *cmd[] = {"not_a_command", NULL};
if (argc > 1)        /* thu them: ./bai8 /etc/passwd */
    cmd[0] = argv[1];
...
if (pid == 0) {
    execvp(cmd[0], cmd);
    perror("execvp");
    _exit(127);
}
```

```
$ ./bai8
execvp: No such file or directory
Child exit code = 127
$ ./bai8 /etc/passwd
execvp: Permission denied
Child exit code = 127
$ ./bai8 true
Child exit code = 0
```

Vì sao mã sau `execvp` chỉ chạy khi thất bại: thành công thì ảnh chương trình cũ (mã, stack, cả lời gọi `perror`) đã bị thay – không còn "chỗ để quay về".

So với shell

`not_a_command` → `bash: command not found`, `$? = 127`; `/etc/passwd` → `Permission denied`, `$? =` **126**. Shell phân biệt "không tìm thấy" (`ENOENT`) và "tìm thấy nhưng không chạy được" (`EACCES`). Đề yêu cầu 127 cho mọi lỗi – mở rộng: `_exit(errno == ENOENT ? 127 : 126)`.

Quên _exit?

Con chạy tiếp mã của cha: `waitpid`, `printf`... – "chương trình chạy hai lần".

Bài 9 – Zombie có thời hạn

```c
#define ZOMBIE_SECONDS 20
...
if (pid == 0)
    _exit(0);                   /* con chet ngay */

printf("Parent PID = %ld, child PID = %ld\n",
       (long)getpid(), (long)pid);
printf("Terminal 2: ps -o pid,ppid,stat,cmd"
       " -p %ld\n", (long)pid);
fflush(stdout);

unsigned left = ZOMBIE_SECONDS;
while (left > 0)
    left = sleep(left);         /* con la Z */

/* waitpid (vong EINTR) */
printf("Reaped child %ld\n", (long)r);
```

Kiểm chứng (output ở mục 6): `STAT = Z+`, `CMD = [bai9] <defunct>`, `PPID` = cha; sau 20 giây `ps -p` không còn dòng nào (mã 1).

- In sẵn lệnh ps cho terminal 2 – đỡ gõ sai PID.
- `fflush` trước khi ngủ: nếu chạy `./bai9 > log`, PID vẫn xuất hiện ngay.
- Thử `kill -9 <PID con>` khi đang Z: không đổi gì. `kill -9 <PID cha>`: zombie biến mất (được nhận nuôi rồi thu dọn).

Bài 10 – Orphan

```
fflush(stdout);
pid_t pid = fork();
...
if (pid == 0) {
    printf("Child PID=%ld, PPID ban dau=%ld\n",
           (long)getpid(), (long)getppid());
    fflush(stdout);
    unsigned left = CHILD_SECONDS;
    while (left > 0)
        left = sleep(left);
    printf("Child PID=%ld, PPID sau %d giay=%ld\n",
           (long)getpid(), CHILD_SECONDS,
           (long)getppid());
    return 0;
}
printf("Parent PID=%ld exits now\n", (long)getpid());
return 0;          /* khong waitpid */
```

Kiểm chứng (output ở mục 6): PPID ban đầu = PID cha (377); sau 3 giây = 273 (`/init` của phiên WSL). Ở phòng lab: `ps -o pid,cmd -p <PPID mới>` – thường `systemd --user` hoặc 1.

- Ghi output vào file rồi `sleep 4; cat`: tránh dòng của con chen vào dấu nhắc.
- Để thấy bằng `pstree`: chạy lại với `CHILD_SECONDS` lớn hơn, rồi `pstree -p -s <PID con>` (`-s`: in cả tổ tiên).

Giả định

"PPID ban đầu" in ngay sau `fork`; nếu cha thoát *trước* cả dòng này, PPID ban đầu đã là người nhận nuôi – hiếm, nhưng có.

Bài 11 – SIGINT (Ctrl+C) có kiểm soát

```c
static volatile sig_atomic_t stop = 0;
static void on_int(int sig) { (void)sig; stop = 1; }

int main(void) {
    if (install(SIGINT, on_int) == -1) {
        perror("sigaction");
        return 1;
    }
    printf("PID = %ld (nhan Ctrl+C de dung)\n",
           (long)getpid());
    fflush(stdout);
    while (!stop)
        sleep(1);
    printf("SIGINT received, exiting\n");
    return 0;
}
```

```
$ ./bai11          (Ctrl+C that qua pseudo-terminal)
PID = 407 (nhan Ctrl+C de dung)
^CSIGINT received, exiting
$ echo $?
0
```

Kiểm chứng: `Ctrl+C` thật; `kill -INT <PID>` từ terminal 2 (hai lần liên tiếp cũng chỉ in một dòng); mã thoát 0 (không phải 130).

- `^C` do terminal *in lại* phím bạn gõ, không phải chương trình.
- `Ctrl+C` gửi SIGINT cho *cả nhóm foreground* – nếu chương trình có con, con cũng nhận.

Bài 12 – SIGTERM: kết thúc sạch

```c
if (install(SIGTERM, on_term) == -1) {
    perror("sigaction");
    return 1;
}
printf("PID = %ld\n", (long)getpid());
fflush(stdout);

while (!stop)
    sleep(1);

/* cho nay moi la noi don dep:
   dong file, xoa file tam, ghi log ... */
printf("Clean shutdown\n");
return 0;
```

```
$ ./bai12 > b12.out &        # terminal 1
$ kill -TERM 396             # terminal 2
$ wait; echo "rc=$?"; cat b12.out
rc=0
PID = 396
Clean shutdown
$ ./bai12 > b12.out & ... kill -KILL 399
bash: line 9:   399 Killed     ./bai12 > b12.out
rc=137
PID = 399                    # khong co "Clean shutdown"
```

Kiểm chứng: `kill -TERM` → mã 0 + "Clean shutdown"; `kill -KILL` → 137, *không* dọn dẹp được; `kill -INT` lên tiến trình nền của script: **treo** (bẫy ở mục 7).

Mở rộng 1 và 2 – bộ đệm và WNOHANG

```
$ ./mr1_stdio_fork; echo
BeforeBefore
$ ./mr1_stdio_fork > out.txt; cat out.txt; echo
BeforeBefore
$ ./mr1_stdio_fork -f > out.txt; od -c out.txt
0000000   B   e   f   o   r   e
0000006
```

Giả định: một chương trình, tuỳ chọn `-f` bật `fflush` (để bước 5) – khỏi phải sửa và biên dịch lại. **Giải thích:** "Before" nằm trong bộ đệm của cha khi `fork` → con có bản sao → hai lần `exit` đẩy hai lần.

```
$ ./mr2_wnohang
Child is still running          (x5)
Child 436 exited, code = 3
```

MR2: con ngủ 5 giây, `_exit(3)` (mã khác 0 để thấy rõ đã giải mã). Kiểm chứng: Valgrind sạch; số dòng 5 hoặc 6 tuỳ lúc con dậy.

Khi nào dùng WNOHANG?

Cha có việc khác phải làm (giao diện, nhận kết nối) – hỏi thăm con định kỳ, hoặc vét con trong vòng lặp `SIGCHLD` (MR3).

Mở rộng 3 – SIGCHLD và sigsuspend

```c
sigset_t block, old;          /* chan TRUOC khi fork */
sigemptyset(&block);
sigaddset(&block, SIGCHLD);
sigprocmask(SIG_BLOCK, &block, &old);

for (int i = 0; i < NCHILD; i++)
    spawn_sleeper(secs[i], 10 + i);  /* 1 s, 3 s */

while (reaped < NCHILD) {
    while (!child_changed)
        sigsuspend(&old);     /* mo chan + ngu: 1 buoc */
    child_changed = 0;        /* xoa co TRUOC khi vet */
    reaped += reap_all();     /* while waitpid(-1,..,
                                 WNOHANG) > 0 */
}
```

Kiểm chứng (output ở mục 7): con 1 s và 3 s $\rightarrow$ 2 lần thức, mỗi lần 1 con; hai con cùng 1 s $\rightarrow$ **1 lần thức, 2 con** (4/4 lần chạy); Valgrind sạch.

- Chặn `SIGCHLD` *trước* `fork`: con chết sớm thì tín hiệu "treo" lại, không lọt vào khe giữa kiểm tra cờ và ngủ.
- `reap_all` kết thúc khi `waitpid` trả 0 (còn con đang chạy) hoặc -1/`ECHILD` (hết con).

Đơn giản hơn mà vẫn đúng

`while (!child_changed) sleep(1);` – trễ tối đa 1 giây, không có khe hở. Chấp nhận được cho bài này.

Kiểm chứng cả bộ bài bằng một script (kỹ năng Lab 1)

```
#!/bin/bash
# check.sh - bien dich, kiem tra hanh vi bang ma thoat va tin hieu
make -s || exit 1
echo -n 'Hello! What is your name?' > Hello.txt
[ "$(./bai5)" = "What is your name?" ]   && echo "Bai 5: OK"
[ "$(./bai6)" = "Child exit code = 7" ]  && echo "Bai 6: OK"
diff <(./bai7) <(ls -l) > /dev/null      && echo "Bai 7: OK"
./bai8 2>/dev/null | grep -q '= 127$'    && echo "Bai 8: OK"
./bai12 > o12.txt & P=$!; sleep 0.5
kill -TERM $P; wait $P; rc=$?
[ $rc -eq 0 ] && grep -q 'Clean shutdown' o12.txt && echo "Bai 12: OK"
[ "$(./mr1_stdio_fork | cat)" = "BeforeBefore" ] && echo "MR1: OK (2 lan)"
[ "$(./mr1_stdio_fork -f | cat)" = "Before" ]    && echo "MR1 -f: OK (1 lan)"
VG="valgrind -q --leak-check=full --track-fds=yes --error-exitcode=9"
for p in bai2 bai3b bai4 bai5 bai6 bai7 bai8 mr2_wnohang mr3_sigchld; do
    $VG --log-file=vg.%p.log ./$p > /dev/null 2>&1 7>&- </dev/null
    [ $? -eq 9 ] && echo "LOI BO NHO: $p"
    cat vg.*.log | grep -A1 'Open file descriptor' | grep -q ' at 0x' \
        && echo "RO RI FD: $p"
    rm -f vg.*.log
done
echo "Valgrind: xong"
```

```
$ ./check.sh
Bai 5: OK
Bai 6: OK
Bai 7: OK
Bai 8: OK
Bai 12: OK
MR1: OK (2 lan)
MR1 -f: OK (1 lan)
Valgrind: xong
```

`vg.%p.log`: mỗi tiến trình (cha, con) một file log.

Bài 12 thử bằng `SIGTERM`, *không* bằng `SIGINT` (tiến trình nền của script bỏ qua `SIGINT`).

Bài 1, 9, 10, 11 cần quan sát bằng mắt ở terminal 2 – không đưa vào script.

Nội dung

1. Mở đầu: từ Lab 5 sang Lab 6
2. Tiến trình, định danh và trạng thái
3. Tạo tiến trình với `fork()`
4. Kết thúc tiến trình và họ hàm exec
5. Chờ tiến trình con: họ `wait`
6. Zombie và orphan
7. Tín hiệu và `sigaction()`
8. Mẫu tổng hợp, lỗi thường gặp và tra cứu
9. Bài tập thực hành
10. Tổng kết

Câu hỏi kiểm tra trong báo cáo

1. Bộ đệm của `FILE *` nằm ở đâu, bao lớn? Vì sao `kill -9` sau `printf("Xong\n")` mất chữ khi ra file mà không mất khi ra terminal?
2. `fork` trả về những giá trị nào, ở tiến trình nào? Viết sai nhánh `-1` thì hậu quả gì?
3. Vì sao bai3a ra pipe thì con luôn in trước? Kết luận gì về "thứ tự cha–con"?
4. Bài 4: vì sao địa chỉ `&idata` giống nhau mà giá trị khác nhau? COW làm gì?
5. Bài 5: cha và con chung gì, riêng gì với fd 3? Nếu con `open` lại file thì sao?
6. Vì sao `printf("Before")` trước `fork` in hai lần *kể cả* ra terminal?
7. `exit` khác `_exit` thế nào? Khi nào con *phải* dùng `_exit`, và khi đó cần làm gì thêm?
8. Vì sao mã sau `execvp` chỉ chạy khi thất bại? Cái gì còn, cái gì mất qua exec?
9. `status = 0x008b` nghĩa là gì? Vì sao không tự tách bit? `_exit(256)` cho cha giá trị nào?
10. Zombie khác orphan thế nào? Ai thu dọn orphan trên máy bạn (ghi PID và tên)?
11. Vì sao handler chỉ được đặt cờ `volatile sig_atomic_t`? Vì sao `while(!f) pause();` có thể treo?
12. Vì sao một lần nhận `SIGCHLD` có thể ứng với nhiều con? Viết vòng thu dọn đúng.

Tiêu chí chấm bài

Đúng đắn – Sạch cảnh báo – Tổ chức hàm – Bền vững – Sạch Valgrind và sạch fd (cả tiến trình con) – Mô hình dữ liệu – Kỷ luật system call (Lab 5) – **Kỷ luật tiến trình** (mới: `fflush` trước `fork`, `_exit` sau `exec` lỗi, mọi con đều được `wait`, giải mã `status` bằng macro, handler chỉ đặt cờ, không giả định thứ tự) – Hiểu biết – Kiểm chứng (output thật của `ps`/`pstree`/`strace`).

Chuẩn bị cho tuần sau: pipe và giao tiếp liên tiến trình

Đọc trước

`Lab7_Tutorial.pdf`: `pipe()`, đóng đầu không dùng, đọc đến EOF, `dup`/`dup2`, pipeline `ls | wc -l`, FIFO; slide lý thuyết SP08 (IPC); `man 2 pipe`, `man 2 dup2`, `man 7 pipe`.

Luyện thêm nếu còn thời gian

- Bài 7: nhận lệnh từ `argv` (`./bai7 date`, `./bai7 ls -a /`) – truyền `argv + 1` cho `execvp`.
- Bài 8: phân biệt 126/127 như shell; in `strsignal(WTERMSIG(s))`.
- Bài 11: bấm `Ctrl+C` hai lần – lần hai thì thoát ngay (gợi ý: đặt lại `SIG_DFL` trong luồng chính).
- Viết `myshell` tối giản: vòng `fgets` → tách `argv` → `fork`/`execvp`/`waitpid`; xử lý EOF (`Ctrl+D`).

Câu hỏi dẫn sang tuần sau

Bài 7 cho `ls -l` in thẳng ra màn hình vì con *kế thừa* fd 1 của cha. Muốn cha *nhận* output đó vào một mảng (để đếm dòng chẳng hạn) thì fd 1 của con phải trỏ đi đâu, và phải sửa nó *trước hay sau* `exec`? Nếu cha đọc mà quên đóng đầu ghi của mình thì `read` trả 0 khi nào? → **Pipe, `dup2` và EOF**.

Phụ lục A – Góp ý cho Lab6_Tutorial.pdf (đã kiểm chứng trên máy)

Mục (tr.)	Tài liệu ghi	Vấn đề và đề xuất sửa
§3.1 (2)	`printf("PID = %ld\\n", ...)`	In ra chữ `\n`, không xuống dòng (chạy thật: `PID = 589\nPPID = 297\n`). Sửa thành `\n`.
§5.2 (4), §11.4 (14)	Biên dịch `gcc -Wall -Wextra`	Với cờ của môn (`-std=c11 -pedantic`) ví dụ `sigaction` lỗi "storage size of 'sa' isn't known", ví dụ `fork` chỉ có `<unistd.h>` lỗi "unknown type name 'pid_t'". Thêm `-D_POSIX_C_SOURCE=200809L` vào lệnh mẫu.
§5.3 (5)	Ví dụ không gian địa chỉ	Không kiểm tra `fork` trả −1, không kiểm tra `waitpid` – trái với §14, lỗi 1. Nên thêm.
§5.6, §6.2 (6)	Con dùng `_exit` để không đẩy lại bộ đệm	Thiếu vế ngược: nếu *con* đã `printf`, `_exit` làm *mất* output đó khi stdout là file/pipe → `fflush` trước `_exit`. Nên thêm `fflush(stdout)` trước `fork` trong mọi ví dụ.
§7.5 (9)	Handler "về mặc định đối với nhiều tín hiệu"	Chính xác: *mọi* tín hiệu đang bắt bằng handler → `SIG_DFL`; tín hiệu đang `SIG_IGN` được *giữ*; mặt nạ tín hiệu giữ nguyên.
§11.3–11.4 (14)	`while (!stop) sleep(1);`	Đúng. Nên nói thêm vì sao không dùng `pause()` (khe hở mất tín hiệu) và giới thiệu `sigsuspend` – Mở rộng 3 cần một cách "ngủ chờ tín hiệu".
§12 (15)	Vét `waitpid`, *sau đó* `child_changed = 0`	Có thể mất một con chết giữa hai bước. Đặt `child_changed = 0` *trước* vòng vét.
§5.2 (4)	"Chạy nhiều lần để thấy thứ tự thay đổi"	Trên Linux hiện nay cha gần như luôn in trước ra terminal (199/200); sinh viên dễ kết luận ngược. Nên nêu thêm ảnh hưởng của bộ đệm (ra pipe: con trước 200/200 ở bai3a).

Phụ lục B – Góp ý cho Lab6_Exercises.pdf và đối chiếu SP07

Exercises: chỗ để ngỏ và quyết định

- Bài 3a: "không `wait` *trước* khi in" → lời giải `waitpid` *sau* khi in (không để orphan, dấu nhắc không chen ngang).
- Bài 4: "hai giá trị khác" không nêu → nhân 3 (3000, 450).
- Bài 8: 127 cho mọi lỗi `exec`; shell dùng 126 cho `EACCES` – nêu trong phân tích.
- Mở rộng 1: đề hỏi terminal khác file thế nào – với `"Before"` *không* `'\n'` hai trường hợp *giống nhau* (đều 2 lần). Nếu muốn thấy khác biệt, đề nên dùng `"Before\n"`.
- Mở rộng 1–3 không đặt tên file → `mr1_stdio_fork.c`, `mr2_wnohang.c`, `mr3_sigchld.c`.
- Phần 4: "`-Wall -Wextra`" → môn dùng thêm `-std=c11 -pedantic -D_POSIX_C_SOURCE=200809L`; đề không yêu cầu `Makefile`, quy trình môn từ Lab 6 có.

SP07 (*Linux Process Handling*): chuyển GV lý thuyết

- `child1.c`/`child2.c`: `void main()`, status chưa khai báo, dấu nháy cong `" "` – không biên dịch.
- `msh.c`: `'\0'` (nháy cong); con dùng `exit(EXIT_FAILURE)` sau `execlp` lỗi – trái với chính khuyến nghị `_exit` của SP07; không kiểm tra `fgets` trả `NULL` → `Ctrl+D` gây lặp vô hạn.
- `orphan.c`: `printf` trước `fork` – chuyển hướng ra file thì dòng "Parent ... exiting" in hai lần; `%d` cho `pid_t`.
- "Week 6 connection: `strcspn`" và "Lab 7 Preview: Tiny Shell" theo giáo trình cũ; giáo trình mới: Lab 7 là pipe/IPC.

Đính chính Lab 5 (slide "Lab 5 trong môn học")

Các dòng "Lab 6: `fgets`+`sscanf`, `ls -l` clone, `du`" và "tiny shell (Lab 7)" theo giáo trình cũ.

Cảm ơn đã lắng nghe!